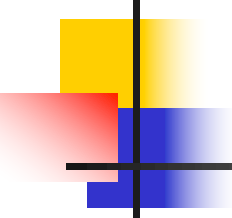


Gestion des exceptions

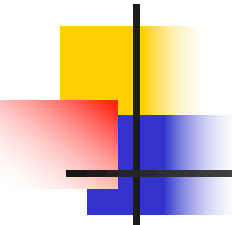


Dr. Khadim Dramé
Khadim.drame@univ-zig.sn
Département Informatique
Université Assane Seck de Ziguinchor



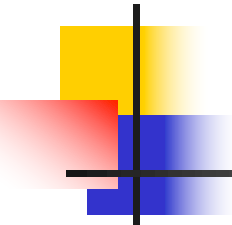
Plan du cours

- Introduction
- Exceptions prédéfinies
- Levée d'une exception
- Interception et traitement d'exceptions
- Définition de nouvelles exceptions
- Exceptions et méthodes
- Structure try-with-resources



Introduction

- **Exception** : **erreur** produite lors de l'**exécution** d'un programme
 - **Interruption** de l'exécution du programme en général
 - Mécanisme de gestion des exceptions séparant leur traitement du programme proprement dit
 - **levée** d'une exception lorsqu'une erreur est détectée
 - **capture** et **traitement** de l'exception ensuite
- ➡ plus de **lisibilité** et **fiabilité** des programmes



Introduction

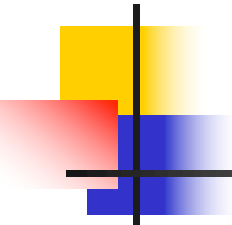
■ Exemple

```
package sn.uasz.m1.exception;  
public class TestException1 {  
    public static void main(String[] args){  
        int x = 5, y = 0;  
        System.out.print(x / y);  
        System.out.println("Fin de calcul");  
    }  
}
```

■ Message affiché par la JVM

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at sn.uasz.m1.exception.TestException1.main(TestException1.java:5)
```

■ Interruption de l'exécution du programme

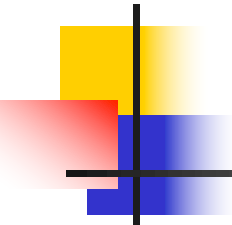


Exceptions prédéfinies

- Trois catégories d'exceptions
 - Classes héritant de la classe **Error**
 - **erreurs critiques** qui ne sont pas censées être gérées en temps normal (**OutOfMemoryError**)
 - Classes héritant de la classe **RuntimeException**
 - erreurs **pouvant éventuellement être gérées** par le programme (**NullPointerException**)
 - Classes héritant de la classe **Exception**
 - **erreurs qui doivent normalement être gérées** par le programme (**EOFException**)
- **Error** et **Exception** héritent de **Throwable**

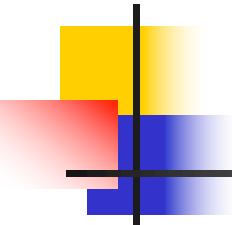
Exception prédéfinies





Exceptions prédéfinies

- Quelques exemples de classes **exception**
 - **FileNotFoundException**: un fichier portant le nom spécifié n'existe pas ou est inaccessible
 - **ArithmeticException**: une opération arithmétique invalide (Exemple : division par 0)
 - **IllegalArgumentException**: signale le passage d'un argument illégal ou incorrect à une méthode
 - **NullPointerException**: une référence **null** est utilisée pour invoquer une méthode ou accéder à un attribut
 - **NumberFormatException**: conversion d'une chaîne ne correspondant pas à un nombre valide



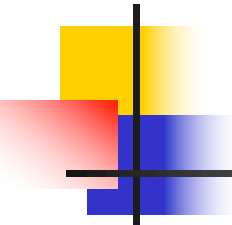
Levée d'une exception

- Pour signaler qu'un événement exceptionnel s'est produit
- Syntaxe

```
throw new nom_exception();
```

```
throw new ArithmeticException();
```

- Lorsqu'une exception est levée (lancée)
 - arrêt de l'exécution normale du programme
 - saut de toutes les instructions jusqu'à ce qu'elle soit interceptée ou qu'on sorte du programme

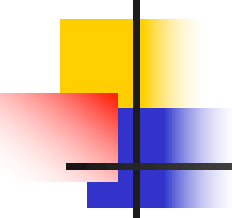


Levée d'une exception

■ Exemple

```
package sn.uasz.m1.exception;
public class TestException1 {
    public static void main(String[] args) throws Exception {
        int n=10;
        System.out.println("instruction 1"); // 1
        if (n==10) throw new Exception();
        System.out.println("instruction 2"); // 2
        System.out.println("instruction 3"); // 3
        System.out.println("instruction 4"); // 4
    }
}
```

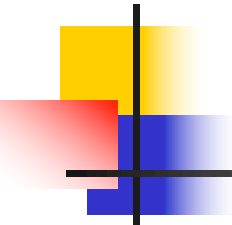
- instructions 2, 3 et 4 pas exécutées



Interception et traitement d'exceptions

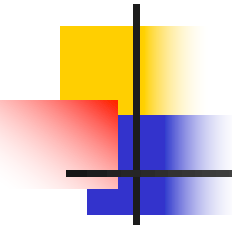
- Syntaxe globale d'interception et de traitement d'une exception

```
try {  
    // Bloc d'instructions pouvant déclencher une erreur  
} catch (Exception exception) {  
    // Bloc d'instructions pour traiter une éventuelle  
    exception  
} finally {  
    // Bloc d'instructions à exécuter dans tous les cas,  
    // que le try se soit exécuté avec succès ou erreur.  
}
```



Bloc try

- Bloc d'instructions **obligatoire**
- Il contient les instructions susceptibles de déclencher une exception
- Si une exception est levée, les **instructions restantes** de ce bloc sont **ignorées**
- Traitement d'une exception lancée dans ce bloc dans les blocs **catch** ou **finally**
- **Un ou plusieurs** blocs **catch** pour un bloc **try**

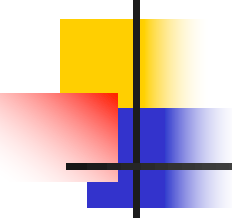


Bloc try

■ Exemple

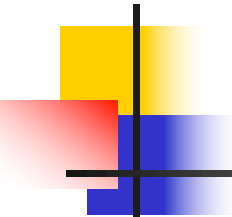
```
package sn.uasz.m1.exception;
public class TestException2 {
    public static void main(String[] args) {
        int n=10;
        try {
            System.out.println ("instruction 1"); // 1
            if (n==10) throw new Exception();
            System.out.println ("instruction 2"); // 2
        } catch (Exception e) {
            System.out.println ("instruction 3"); // 3
        }
        System.out.println ("instruction 4"); // 4
    }
}
```

- instruction 2 pas exécutée



Bloc catch

- Un ou plusieurs blocs **catch** pour un bloc **try**
- Chaque bloc **catch** pour capturer et traiter une exception particulière
- Si l'exception lancée correspond à un bloc catch, ce bloc est exécuté et les autres ignorés
- Cas de plusieurs blocs catch
 - les exceptions associées doivent être **ordonnées de la plus spécifique à la plus générale**

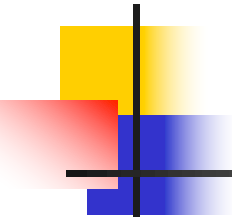


Bloc catch

■ Exemple

```
package sn.uasz.m1.exception;
import java.io.FileNotFoundException;
public class TestException3 {
    public static void main(String[] args) throws Exception {
        int n=10;
        try {
            System.out.println ("instruction 1"); // 1
            if (n==10) throw new Exception();
            System.out.println ("instruction 2"); // 2
        } catch (FileNotFoundException e) {
            System.out.println ("instruction 3"); // 3
        }
        System.out.println ("instruction 4"); // 4
    }
}
```

- instructions 2, 3 et 4 pas exécutées

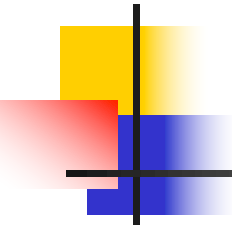


Bloc catch

■ Exemple

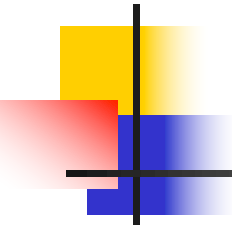
```
package sn.uasz.m1.exception;
import java.io.FileNotFoundException;
public class TestException4 {
    public static void main(String[] args) throws Exception {
        int n=10;
        try {
            System.out.println ("instruction 1"); // 1
            if (n==10) throw new Exception();
            System.out.println ("instruction 2"); // 2
        } catch (FileNotFoundException e1) {
            System.out.println ("instruction 3"); // 3
        } catch (Exception e2) {
            System.out.println ("instruction 4"); // 4
        }
        System.out.println ("instruction 5"); // 5
    }
}
```

- instructions 2 et 3 pas exécutées



Bloc finally

- Un bloc **finally** peut être déclaré à la suite des blocs **catch**
- Il est **optionnel** mais est exécuté systématiquement s'il est défini
- Généralement utilisé pour libérer les ressources ouvertes (un fichier, une connexion, etc.) dans un bloc **try**



Bloc finally

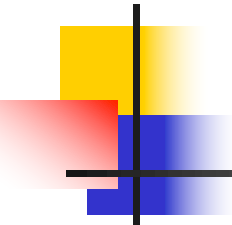
■ Exemple

```
package sn.uasz.m1.exception;

public class TestException5 {

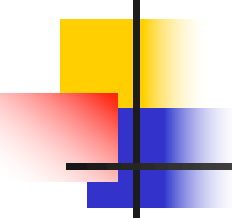
    public static void main(String[] args) throws Exception {
        try {
            String chaine="bonjour";
            System.out.println("Avant conversion"); // 1
            int i=Integer.parseInt(chaine);
            //Levée d'une exception de type NumberFormatException
            System.out.println("Après conversion"); // 2
        } finally {
            System.out.println("On passe par le bloc finally"); //3
        }
    }
}
```

■ instructions 1 et 3 exécutées



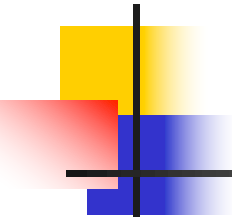
Définition de nouvelles exceptions

- Un grand nombre de classes d'exception représentant différents types d'erreurs
- Mais besoin de mettre en œuvre de nouvelles exceptions
- Définition d'une nouvelle classe de type exception:
 - ➡ étendre une classe exception déjà définie
 - En général, extension de la classe `Exception`



Définition de nouvelles exceptions

```
package sn.uasz.m1.exception;  
public class NouvelleException extends Exception {  
    public NouvelleException(){}  
    public NouvelleException(String message){  
        super(message);  
    }  
}
```



Exceptions et méthodes

- Déclaration **obligatoire** des exceptions **vérifiées** susceptibles d'être levées dans une méthode sans être traitées
- Syntaxe

```
throws nom_exception
```

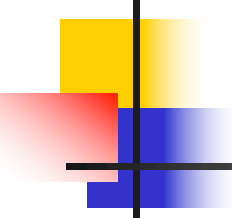
```
float division(float a, float b) throws ArithmeticException{...}
```

- Dans le cas de plusieurs exceptions, les séparer par une virgule
- Déclaration pas obligatoire pour les classes **Error** et **RuntimeException**



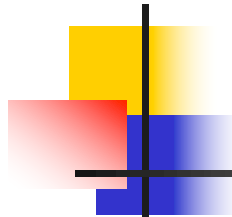
Exceptions et méthodes

```
class ValeurNonPermiseException extends Exception {} //classe exception
class Factorielle {
    static int fact(int n) throws ValeurNonPermiseException { // déclaration
        int res = 1;
        if (n<0){
            throw new ValeurNonPermiseException();
        }
        for(int i = 1; i <= n; i++) {
            res = res * i;
        }
        return res;
    }
    public static void main (String [] args) {
        int x;
        // saisie de la valeur de x
        try {
            System.out.println(fact(x));
        } catch (ValeurNonPermiseException e) {
            System.out.println("Factorielle de "+x+" pas définie !");
        }
    }
}
```



Syntaxe try-with-resources

- Ajoutée depuis la version 7.0 de Java SE
- Un bloc `try` déclare une ou plusieurs ressources
- Fermeture obligatoire de toute ressource à la fin du programme
- La syntaxe `try-with-resources` assure que chaque ressource sera fermée automatiquement
- Pas besoin d'utiliser `finally` pour fermer la ressource



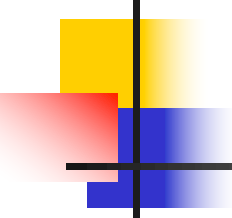
Syntaxe try-with-resources

■ Exemple

```
BufferedReader reader = new BufferedReader(new
FileReader("fichier.txt"))
try {
    String ligne=null;
    while ((ligne = reader.readLine()) != null) {
        System.out.println(ligne);
    }
} finally {
    // on ferme la ressource
}
```

//Avec try-with-resources

```
try (BufferedReader reader = new BufferedReader(new
FileReader("fichier.txt"))) {
    String ligne=null;
    while ((ligne = reader.readLine()) != null) {
        System.out.println(ligne);
    }
} // fermeture automatique de la ressource
```



Exercice d'application

- Écrire un programme Java avec une méthode de classe *saisie()* permettant de saisir correctement un entier.
 - Si l'utilisateur saisit une donnée dont le format n'est pas celui d'un entier, le programme lève une exception de type **java.util.InputMismatchException** ;
 - Si l'utilisateur saisit un entier négatif, le programme lève une exception de type **NegativeNombreException** (à définir) ;
 - Si la saisie est incorrecte, un message d'erreur est affiché avec la proposition d'effectuer une nouvelle saisie.